# WorkNest: Technical Q&A

117 simple technical questions about the WorkNest project, with short answers, grouped by topic. Use them to prepare for presentation questions.

## The project in general

**1. What is WorkNest?**

A job board and recruitment platform. Candidates find and apply for jobs; a recruitment team manages jobs, client companies and applicants from an admin dashboard.

**2. What kind of application is it?**

A full-stack web application: a React frontend in the browser, an Express REST API on a server, and a MongoDB database.

**3. What is the tech stack in one line?**

React + TypeScript + Vite + Tailwind CSS on the frontend; Node.js + Express + Mongoose + MongoDB on the backend.

**4. Why is it split into two projects?**

The frontend and the API can be developed, deployed and scaled separately, and another client such as a mobile app could reuse the same API.

**5. What does “full-stack” mean here?**

The team built both the part users see (frontend) and the part that stores and protects data (backend and database).

**6. Who are the users of the system?**

Visitors (browse jobs), candidates (apply and track applications), and recruiters and admins (manage the hiring process).

**7. What language is the whole project written in?**

TypeScript, on both the frontend and the backend.

## Frontend basics

**8. What is React?**

A JavaScript library for building user interfaces out of reusable components.

**9. Which version of React is used?**

React 19.

**10. What is a component?**

A reusable piece of UI written as a function that returns JSX, for example `JobCard`, `Navbar` or `Stepper`.

**11. What is JSX?**

HTML-like syntax inside JavaScript/TypeScript that React turns into real page elements.

**12. What are props?**

Inputs passed into a component, like `<JobCard job={job} />`.

**13. What is state?**

Data a component remembers between renders, created with `useState`, such as form inputs, filters and loading flags.

**14. What does useEffect do?**

Runs code after a component renders. WorkNest uses it to load data from the API when a page opens.

**15. What does useMemo do?**

Remembers a calculated value until its inputs change. The admin Candidates, Clients, Jobs and Pipeline pages use it to filter lists without recalculating on every render.

**16. What is React Context?**

A way to share data with every component without passing props down. WorkNest has `AuthContext` (who is logged in) and `SavedJobsContext` (which jobs are bookmarked).

**17. What is a single-page application (SPA)?**

A site that loads one HTML page and then swaps content with JavaScript, so moving between pages doesn't reload the browser.

**18. What is React Router used for?**

Mapping URLs to pages, for example `/find-jobs` to the Find Jobs page. All routes are in `src/App.tsx`.

**19. What is a protected route?**

A page only logged-in users can open. `ProtectedRoute` sends logged-out users to `/login` and non-admins away from `/admin`.

**20. What is Vite?**

The build tool and development server. It reloads the page instantly while you code and bundles the app for production.

**21. What port does the frontend run on?**

`5173` (`http://localhost:5173`).

**22. What is Tailwind CSS?**

A CSS framework of small utility classes, such as `rounded-full` or `text-[14px]`, written directly on elements instead of in separate CSS files.

**23. What is axios?**

A library for making HTTP requests. WorkNest has one shared client in `src/lib/apiClient.ts` that adds the login token to every request.

**24. What are toasts and which library shows them?**

The small pop-up success and error messages, shown with react-toastify.

**25. Which icon library is used?**

react-icons, mainly the Feather set (`FiUser`, `FiBriefcase`).

**26. Where does the frontend store the login token?**

In the browser's `localStorage` under the key `worknest_token`.

**27. What are environment variables on the frontend?**

`VITE_API_URL` (where the API is) and `VITE_GOOGLE_CLIENT_ID`. Vite only exposes variables that start with `VITE_` to the browser.

### 28. Is the job search done on the frontend or the backend?

The frontend. The server sends all open jobs once and the browser filters them as you type.

### 29. Why put search terms in the URL?

So a search like `?q=designer` can be bookmarked or shared and survives a page refresh.

## Backend basics

### 30. What is Node.js?

A runtime that lets JavaScript run on a server instead of in a browser.

### 31. What is Express?

A web framework for Node.js that handles routes, middleware and responses. WorkNest uses Express 5.

### 32. What port does the server run on?

`5000` by default, or the `PORT` environment variable.

### 33. What is a REST API?

An API where each URL is a resource and the HTTP method says the action: GET reads, POST creates, PATCH updates, DELETE removes.

### 34. What format does the API send and receive?

JSON.

### 35. What is a route?

A URL and method paired with the code that handles it, for example `GET /api/jobs` in `routes/jobRoutes.ts`.

### 36. What is a controller?

The function that does the work for a route: reads the request, queries the database, and sends the response.

### 37. What is middleware?

A function that runs before the controller, for example to check the login token or read an uploaded file.

### 38. What does express.json() do?

Reads JSON request bodies so controllers can use `req.body`.

### 39. What is CORS and why is it needed?

A browser rule that blocks calls to a different address. The frontend (port 5173) and API (port 5000) are different addresses, so the server allows them with the `cors` library.

### 40. What is dotenv for?

Loading secret settings from the `.env` file into `process.env`.

### 41. What is the health check endpoint?

`GET /api/health` returns `{"status":"ok"}` to confirm the server is running.

### 42. What is tsx?

A tool that runs TypeScript directly. `npm run dev` uses `tsx watch` to restart the server on every save.

### 43. What does npm run build do on the server?

Compiles TypeScript into JavaScript in `dist/`; `npm start` then runs `dist/server.js`.

### 44. Why does the server connect to the database before listening?

So it never accepts requests it can't answer. If the connection fails, the process exits.

## Database

### 45. Which database is used?

MongoDB, hosted on MongoDB Atlas.

### 46. Is MongoDB SQL or NoSQL?

NoSQL. It stores JSON-like documents in collections instead of rows in tables.

### 47. What is Mongoose?

A library that defines schemas (models) for MongoDB and validates data before saving.

### 48. How many models are there, and what are they?

Six: User, Company, Job, Application, SavedJob and Notification.

### 49. How are models connected?

By references: a record stores another record's id. For example, a Job stores its Company's id, and an Application stores its Job and User ids.

### 50. What does .populate() do?

Replaces a stored id with the full referenced record, like a job's company name and logo.

### 51. What does .lean() do?

Returns plain JavaScript objects instead of full Mongoose documents: faster, but schema defaults aren't applied.

### 52. What are timestamps in a schema?

Automatic `createdAt` and `updatedAt` fields on every record.

### 53. What is a slug?

A URL-friendly version of a name, like `frontend-engineer`, stored on jobs and companies.

### 54. How do you stop someone applying to the same job twice?

A unique index on `{job, applicant}` in the Application model. It only applies to logged-in users, so guests can still apply.

### 55. What are the application stages?

submitted, review, shortlisted, interview, offer, hired, and rejected.

### 56. What job statuses exist?

draft, open, closed and archived.

### 57. What is the seed script?

`npm run seed` wipes and refills the jobs and companies with sample data. `npm run seed-candidates` adds demo candidates.

## Authentication and security

### 58. How does a user log in?

They send email and password to `POST /api/auth/login`; if correct, the server returns a token.

**59. What is a JWT?**

A JSON Web Token: a signed string that proves who the user is. WorkNest's contains the user's id and role.

**60. How long does a token last?**

7 days by default (`JWT_EXPIRES_IN`).

**61. How is the token sent to the server?**

In the `Authorization: Bearer <token>` header, added by axios.

**62. How are passwords stored?**

As bcrypt hashes with a cost of 10. The real password is never saved.

**63. What is hashing?**

A one-way scramble: you can check a guess against the hash, but you can't turn the hash back into the password.

**64. How does email verification work?**

After sign-up the server emails a 6-digit code. It expires in 10 minutes and allows 5 wrong attempts.

**65. Why is the email code hashed too?**

So a leaked database doesn't reveal codes that could be used to take over accounts.

**66. How does Google sign-in work?**

The Google popup returns an access token. The server checks it with Google and confirms it was issued for WorkNest's Client ID before logging the user in.

**67. What are the user roles?**

candidate, recruiter and admin.

**68. What does the protect middleware do?**

Rejects requests with no token, or an expired or invalid one, with 401.

**69. What does the authorize middleware do?**

Rejects users without the right role with 403, even if their token is valid.

**70. What does the identify middleware do?**

Reads the token if there is one but doesn't require it. Used for applying, so guests can apply too.

**71. What is the difference between 401 and 403?**

401 means “we don't know who you are” (not logged in). 403 means “we know who you are, but you're not allowed”.

**72. Is hiding admin pages on the frontend enough?**

No. Anyone can call the API directly, so the real check is on the server with `authorize`.

**73. How do you stop a user making themselves an admin?**

The profile update only accepts a fixed list of fields; role, email and verification can't be changed through it.

**74. What does select: false do on the password field?**

Leaves it out of every query result unless the code asks for it explicitly.

### 75. Where are secrets kept?

In `.env` files, which are git-ignored so they never reach GitHub.

### 76. How do you make someone an admin?

`npm run make-admin their@email.com` in the server folder.

## Features and how they work

### 77. What does the Apply page do?

A 4-step wizard: your details, CV and links, screening questions, review. It pre-fills from the candidate's profile.

### 78. What are screening questions?

Custom questions an admin adds to a job; candidates must answer all of them to apply.

### 79. Can a guest apply without an account?

Yes. The application is saved without a linked user.

### 80. How does saving a job work?

A SavedJob record links the user and the job. `SavedJobsContext` keeps the saved ids so bookmark icons fill in instantly.

### 81. How are “closes in X days” values calculated?

The server stores a closing date and works out the days left, rounded up. Jobs past their date are hidden from listings.

### 82. What notifications exist?

Application status changes, a saved job closing soon, and new job matches.

### 83. What does the admin Overview show?

Headline stats, recent applications, applications by status, jobs closing soon, most-applied roles and a pipeline snapshot.

### 84. What is the Pipeline page?

Where staff review an application: move it between stages, score it on a 0 to 5 scorecard, and leave internal notes.

### 85. How is a candidate's status (Placed, Employed, Open to work) decided?

Placed if any application is marked hired; Employed if the candidate set `openToWork` to false; otherwise Open to work.

### 86. What does the profile completion score do?

Shows how much of the profile is filled in, encouraging candidates to finish it.

## Files and emails

### 87. Where are CVs and logos stored?

On Cloudinary, a cloud file service. The database only stores the link.

### 88. What is multer?

Middleware that reads uploaded files from a form submission.

### 89. Why keep uploads in memory instead of on disk?

The file goes straight to Cloudinary, so nothing is left on the server.

### 90. What file types and sizes are allowed?

CVs: PDF, DOC or DOCX up to 5 MB. Logos: PNG, JPEG, WEBP or SVG up to 2 MB.

### 91. Which service sends emails?

Brevo, through its API.

### 92. Which emails are sent?

Verification code, password reset code, welcome email, application received, and application status changes.

### 93. What happens if email isn't set up?

Sign-up still works: in development the code is printed in the server terminal.

## Tools, setup and deployment

### 94. What is TypeScript and why use it?

JavaScript with types. It catches mistakes, like a missing field, before the code runs.

### 95. What is ESLint?

A tool that scans code for bugs and bad patterns. Run it with `npm run lint`.

### 96. What is npm?

Node's package manager. It installs libraries listed in `package.json` and runs scripts.

### 97. What is package.json?

The file listing a project's libraries (dependencies) and its scripts.

### 98. What is the difference between dependencies and devDependencies?

Dependencies are needed to run the app; devDependencies are only needed while developing, like TypeScript and ESLint.

### 99. What is .gitignore?

A list of files git should never track, such as `node_modules` and `.env`.

### 100. What is Git and GitHub used for?

Version control and hosting the code. There are two repos: `WorkNestFront` and `WorkNestBack`.

### 101. How do you run the project locally?

`npm install`, then `npm run dev`, in both folders. The server must be running for logins and data.

### 102. Where is the frontend hosted?

Vercel. `vercel.json` sends every URL to `index.html` so React Router can handle it.

### 103. Why does a SPA need that rewrite?

Without it, refreshing on `/find-jobs` would look for a file that doesn't exist and show a 404.

### 104. Which environment variables does the server need?

`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `GOOGLE_CLIENT_ID`, `CLIENT_ORIGIN`, `PORT`, plus the Brevo and Cloudinary keys.

## HTTP status codes used

### 105. 200 OK

The request worked, for example a list of jobs.

### 106. 201 Created

Something new was saved, for example an application or a client.

### 107. 400 Bad Request

Invalid input, such as a missing field or a malformed id.

### 108. 401 Unauthorized

Not logged in, or the token expired.

### 109. 403 Forbidden

Logged in, but the role isn't allowed.

### 110. 404 Not Found

The record doesn't exist, such as a deleted job.

### 111. 500 Internal Server Error

Something unexpected failed on the server; it's logged in the terminal.

## Design decisions and improvements

### 112. Why MongoDB instead of a SQL database?

Job and profile data varies in shape (skills lists, screening answers), which fits documents well; Mongoose still enforces a structure.

### 113. Why client-side search?

With a few hundred jobs it's instant and needs no extra requests. With thousands, search would move to the server with pagination.

### 114. Why Tailwind instead of normal CSS files?

Styles sit next to the markup, so components are self-contained and fast to build.

### 115. Why TypeScript on both sides?

One language across the project, and types catch mismatches between what the API sends and what the UI expects.

### 116. What was a bug you fixed?

Older jobs had no screening-questions field, and `.lean()` skips defaults, so the Apply page crashed. The API now always sends an empty list.

### 117. What would you improve next?

Server-side search and pagination, profile photos, automated tests, and recruiter accounts limited to their own clients.